

Paradigmas y Lenguajes de Programación

INFORME: TRABAJO PRÁCTICO INTEGRADOR 2026

Sistema de Semáforos Inteligentes y Análisis Comparativo de Paradigmas



GRUPO 4: OCaml

Integrantes:

- Torres Yleana Beatriz 44.982.909
- Almiron José 46.602.154
- Lagraña Balzaretti Juan Martín 46.523.195
- López Nicolás Jonatan 40.509.009

JUNIO 2026

Contenido

INDICE	2
INTRODUCCIÓN	3
DESARROLLO	4
OCAML.....	7
BITÁCORA DE DEPURACIÓN.....	12
CONCLUSION	24
Sobre OCalm.....	24
Sobre el proyecto en general	25
BIBLIOGRAFÍA	27

INTRODUCCIÓN

El presente informe documenta el desarrollo del **Trabajo Práctico Integrador 2026** de la cátedra de **Paradigmas y Lenguajes de Programación** de la carrera Licenciatura en Sistemas de Información de la **FaCENA – UNNE**.

El trabajo consiste en el modelado de un **Sistema de Semáforos Inteligentes** implementado en **Common Lisp**, con un análisis comparativo paralelo en **OCaml**, lenguaje asignado al grupo por sorteo entre diversas alternativas funcionales. Los grupos de trabajo fueron conformados de manera aleatoria, lo que sumó al desafío técnico la experiencia de coordinar y colaborar con compañeros no elegidos previamente. En él se incorporan y profundizan los conceptos trabajados a lo largo de la cursada sobre **recursividad** y el **paradigma funcional**, aplicándolos sobre un problema concreto que exigió explorar con mayor detalle aspectos como la **pureza de funciones**, la **inmutabilidad** y el manejo de **efectos secundarios**.

Como herramienta de trabajo colaborativo se utilizó **GitHub**, donde se encuentra disponible no solo el código fuente de ambas implementaciones, sino también toda la documentación asociada al proyecto, reflejando el proceso de desarrollo y toma de decisiones del grupo a lo largo del trabajo.

DESARROLLO

Desarrollo de los fundamentos del diseño

La mayoría de las decisiones de diseño de cada función ya quedaron documentadas directamente en el código, en el encabezado de comentarios que clasifica su naturaleza, estrategia e impacto en memoria, además de las observaciones puntuales que se fueron dejando debajo de cada implementación. Por eso, más que repetir esa información, en esta sección se busca explicar el *por qué* de esas clasificaciones y de las decisiones que se tomaron como grupo, especialmente en los puntos donde la consigna dejaba margen de interpretación.

Un criterio transversal que guio todo el desarrollo fue evitar el hardcodeo de valores siempre que la consigna lo permitiera, priorizando funciones reutilizables y parametrizadas por sobre soluciones específicas para las reglas de negocio actuales (rojo=90, verde=120, amarillo=6). En los casos donde el hardcodeo fue inevitable, porque la consigna fijaba explícitamente la forma de los parámetros de entrada, se dejó constancia en el comentario de la función, aclarando qué se hubiese preferido hacer de no existir esa restricción.

Transición

Se implementó como función pura mediante `cond`, evaluando el par (color-actual, cambiar-a) contra la única secuencia cíclica válida del semáforo (rojo→verde, verde→amarillo, amarillo→rojo), confirmada con el docente tal como se detalla en la bitácora. Se evaluó extraer la construcción de la lista de retorno a una función auxiliar usando `format` para reducir la repetición de las tres ramas casi idénticas, pero se decidió como grupo mantener la versión explícita por sobre la abstracción, priorizando la legibilidad y que cada caso quede a la vista de forma directa.

Timer

Función pura que, a partir de un timestamp Unix, calcula el color vigente mediante `mod` sobre la duración total del ciclo (216 segundos) y un `cond` que evalúa en qué tramo cae ese resto. Se decidió que el ciclo arranca en rojo en el instante 0 de la época Unix y se repite indefinidamente (incluso para timestamps negativos, gracias a `mod`), siguiendo la respuesta del docente de que el equipo podía decidir el origen del ciclo. El `shadow 'timer` que aparece antes de la definición es exclusivamente una solución técnica para el conflicto de nombres con SBCL (detallado en la bitácora), no una decisión de diseño funcional.

Auditoría

Función impura (su único propósito es imprimir) que se diseñó para recibir un solo parámetro: el timestamp actual. El color anterior se obtiene internamente reutilizando timer con (- timestamp 1), en lugar de pedirle al que llama que calcule o pase ambos colores. Esta decisión, explicada en la bitácora, busca una interfaz mínima: como timer es pura y el ciclo es determinístico, no hay motivo para duplicar esa responsabilidad afuera de la función.

Duracion-ciclo

Función pura y mínima: simplemente suma los tres tiempos recibidos como parámetros. Se decidió que reciba rojo, verde y amarillo como argumentos (y no valores fijos) precisamente para que pueda ser reutilizada por otras funciones (como distribucion-hora) bajo distintas reglas de negocio, no solo las actuales.

Recomendacion-ciclo

Función impura por cond que clasifica la duración recibida en tres rangos (menor a 35, mayor a 150, o el rango recomendado), siguiendo directamente los criterios de psicología del conductor indicados en la consigna. No presentó mayor complejidad de diseño más allá de mapear esos rangos a los tres mensajes pedidos.

Ciclos-por-tiempo

Acá sí fue inevitable un hardcodeo: la consigna especifica que la función recibe únicamente la cantidad de minutos, por lo que no hay forma de parametrizar la duración del ciclo (216) sin agregar argumentos que la consigna no permite. Se dejó esto explícito en el comentario de la función, indicando que de haber podido recibir rojo/verde/amarillo se hubiera preferido reutilizar duracion-ciclo en lugar de fijar el 216 directamente, igual que se hizo en distribucion-hora.

Repartir-resto

Esta función auxiliar nació de un problema concreto de distribucion-hora (detallado en el punto 7 y 8 de la bitácora): el reparto del tiempo sobrante de un ciclo entre las distintas fases es la misma operación aplicada secuencialmente sobre una lista. Se decidió extraerla como función pura y recursiva de cola, parametrizada por una lista de duraciones de longitud arbitraria, para que no dependiera de cuántas fases tuviera el semáforo. Esto fue clave para que después distribucion-hora2 (con el doble de fases por la intermitencia) pudiera reutilizarla sin modificarla.

Distribucion-hora

Función impura que combina cálculo aritmético con mapcar. Se decidió que reciba rojo, verde y amarillo como parámetros (y no como valores fijos), tal como exige la consigna al hablar de "ciertas reglas de negocio o las actuales", para que la función sirva para cualquier configuración del semáforo. La cantidad de ciclos completos en una hora se

calcula reutilizando duracion-ciclo en lugar de un valor fijo, evitando así el hardcodeo del 216 (a diferencia de ciclos-por-tiempo, donde no fue posible). El reparto del resto se delega a repartir-resto, y los totales y la impresión final se obtienen con dos mapcar que combinan en paralelo las listas de duraciones/extras y de nombres/totales, evitando variables intermedias nombradas a mano para cada fase.

Extensión 1: Transicion2, Timer2, Auditoria2, etc.

Para la segunda iteración, se decidió mantener exactamente la misma estrategia que en la versión 1, agregando los tres estados intermitentes (rojo-intermitente, verde-intermitente, amarillo-intermitente) de 3 segundos cada uno como parte del ciclo, que pasa de 216 a 225 segundos. Transicion2 extiende la secuencia válida a seis pasos (rojo → rojo-intermitente → verde → verde-intermitente → amarillo → amarillo-intermitente → rojo), y timer2/auditoria2/recomendacion-ciclo2/duracion-ciclo2/repartir-resto2/distribucion-hora2 son, en su mayoría, adaptaciones directas de sus versiones de la primera iteración, cambiando únicamente la cantidad de fases y sus duraciones. Distribucion-hora2 es el ejemplo más claro de por qué se invirtió esfuerzo en la abstracción de repartir-resto: gracias a que esa función ya era genérica respecto a la longitud de la lista de duraciones, pasar de 3 a 6 fases no implicó reescribir la lógica de reparto, solo ampliar las listas de duraciones y nombres que se le pasan a mapcar.

Si bien la consigna de la Extensión 1 no pedía explícitamente actualizar los casos de prueba del Requerimiento 7 para los nuevos estados intermitentes, decidimos como grupo agregarlos igual para transicion2, timer2, auditoria2, etc., por una cuestión de consistencia: todas las funciones de la primera iteración tienen su batería de casos normal/alternativo/error, y nos pareció que dejar las versiones "2" sin esa misma cobertura rompía la uniformidad del informe y dificultaba verificar que el comportamiento con intermitencia fuera el esperado.

Extensión 2: Informe / Informe2 y funciones auxiliares (universal-a-unix, ejecutar-sistema)

Informe se implementó como función impura que escribe en archivo (con with-open-file y :if-exists :append), reutilizando la misma lógica de comparación de auditoria pero dirigiendo la salida a un archivo en lugar de la terminal, y agregando el formato de fecha legible mediante **local-time** (ya integrado desde la Fase 2). Para poder ejecutar el sistema "en tiempo real" se agregaron dos funciones no solicitadas explícitamente por la consigna: universal-a-unix, que convierte el tiempo interno de Common Lisp a epoch Unix (necesario para que local-time formatee correctamente la fecha actual), y ejecutar-sistema, que mediante recursión de cola llama a informe cada segundo durante una cantidad de segundos dada, respetando la restricción de no usar bucles imperativos (loop, dotimes, etc.).

Al igual que con la intermitencia, decidimos agregar casos de prueba para informe, informe2, universal-a-unix, ejecutar-sistema y ejecutar-sistema2 aunque la consigna no lo pidiera de forma explícita para estas funciones extra, por el mismo criterio de consistencia con el resto del Requerimiento 7: si todas las funciones del sistema tienen su normal/alternativo/error documentado, nos pareció más prolijo, y más útil para quien corrija, que las funciones agregadas por nosotros también lo tuvieran, en lugar de dejarlas como una excepción sin verificar.

OCAML

Presentación del lenguaje

OCaml es un lenguaje de programación funcional, de tipado estático y maduro que combina potencia y pragmatismo, siendo ideal para construir sistemas de software complejos. Fue creado en 1996 por Xavier Leroy, Jérôme Vouillon, Damien Doligez y Didier Rémy en el instituto de investigación INRIA, en Francia.

Entre sus características principales se destacan la recolección de basura generacional para la gestión automática de memoria, funciones de primera clase, verificación estática de tipos, inferencia de tipos, polimorfismo paramétrico, programación inmutable, tipos de datos algebraicos y coincidencia de patrones.

Industrias y áreas de uso

Gracias a sus sólidas características de seguridad y alto rendimiento, muchas empresas utilizan OCaml para mantener sus datos operando de forma segura y eficiente. Se aplica principalmente en:

- **Finanzas y trading algorítmico**
- **Ciberseguridad**
- **Infraestructura tecnológica y DevOps**
- **Redes sociales y tecnología web**
- **Investigación y academia**

La página oficial de OCaml presenta casos de estudio concretos que ilustran la diversidad de áreas en las que se aplica el lenguaje. Entre ellos se destacan: **Ahrefs**, que lo utiliza para rastreo web y procesamiento de datos a escala de petabytes; **HyPer**, una plataforma de análisis de sensores y automatización para agricultura sostenible; **Be Sport**, una red social para deportes y atletas; **LexiFi**, que lo emplea para desarrollar un lenguaje de modelado financiero; **Nitrokey**, que construyó con OCaml y MirageOS un módulo de seguridad de hardware abierto (NetHSM); e **Imandra**, que lo usa para cumplimiento financiero con razonamiento automatizado. Además, **Parsimoni** desarrolló SpaceOS, un sistema operativo multiusuario para satélites; **Robur** lo aplica para servicios de internet seguros; **Terrateam** lo usa en una plataforma de

infraestructura como código; y **VCast** lo emplea en una plataforma de votación en línea segura y verificable.

Esto demuestra que OCaml se aplica en industrias tan variadas como las finanzas, la ciberseguridad, la agricultura, el sector aeroespacial, la tecnología electoral y las redes sociales.

Empresas que lo utilizan

Entre las empresas destacadas que usan OCaml se encuentran:

- **Jane Street:** utiliza OCaml para su sistema de trading a gran escala, con más de 500 programadores OCaml y 30 millones de líneas de código.
- **Facebook:** ha construido importantes herramientas de desarrollo usando OCaml.
- **Docker:** utiliza OCaml en su suite de tecnología integrada para desarrollo y operaciones de IT.
- **Bloomberg:** líder en información financiera y de negocios a nivel global, usa OCaml en sistemas críticos.
- **Microsoft y Ahrefs** también forman parte de la comunidad industrial de OCaml.

Para consultar el listado completo de empresas que utilizan OCaml, puede accederse al sitio oficial del lenguaje en la sección dedicada a usuarios industriales: <https://ocaml.org/industrial-users/businesses>. A modo ilustrativo, se adjunta una captura de pantalla de dicha página oficial, donde se pueden observar algunas de las empresas más reconocidas que emplean OCaml en sus sistemas, entre ellas Facebook, Microsoft, Docker, Jane Street, Bloomberg, Ahrefs, Wolfram y Dassault Systèmes.



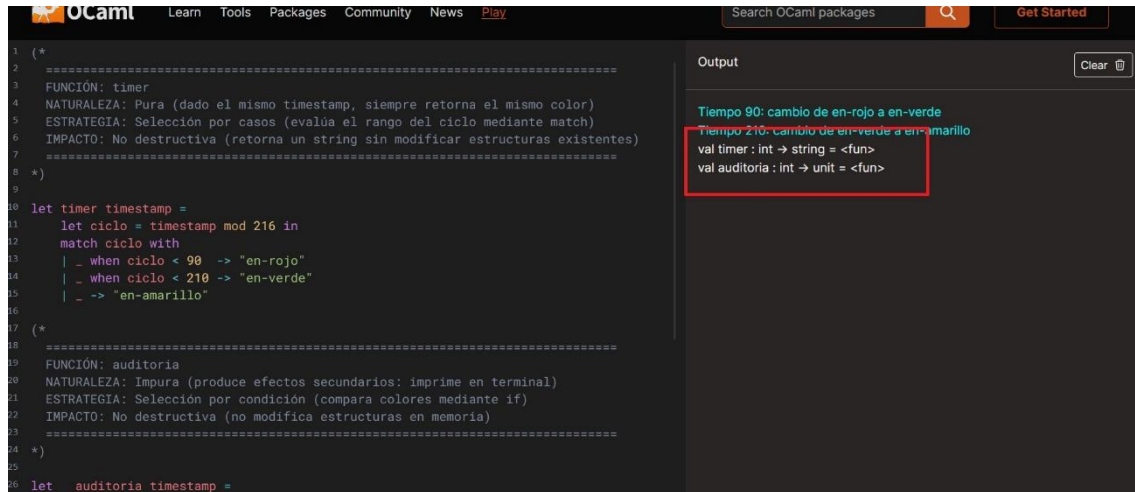
Preguntas respecto a OCaml

1. Al igual que Haskell, OCaml usa tipos estáticos pero cuenta con Inferencia de Tipos. ¿Tuvieron que declarar explícitamente de qué tipo eran las funciones o el compilador lo dedujo solo? Muestren cómo lo interpreta el entorno.

2. OCaml es un lenguaje "orientado a expresiones". ¿Cómo cambia esto la estructura de control de flujos (como el uso de match...with) comparado con los condicionales de Lisp?

Pregunta 1

En OCaml no fue necesario declarar los tipos explícitamente. El compilador los dedujo solo mediante inferencia de tipos. Esto se puede ver en el output del playground donde muestra:



The screenshot shows the OCaml playground interface. On the left, there is OCaml code defining two functions: 'timer' and 'auditoria'. The 'timer' function uses a 'match' expression to return a string based on a timestamp. The 'auditoria' function prints a message. On the right, the 'Output' panel shows the inferred types for these functions, which are highlighted with a red box. The output indicates that 'timer' has the type 'int -> string' and 'auditoria' has the type 'int -> unit'.

```
1 (*  
2 =====  
3 FUNCIÓN: timer  
4 NATURALEZA: Pura (dado el mismo timestamp, siempre retorna el mismo color)  
5 ESTRATEGIA: Selección por casos (evalúa el rango del ciclo mediante match)  
6 IMPACTO: No destructiva (retorna un string sin modificar estructuras existentes)  
7 =====  
8 *)  
9  
10 let timer timestamp =  
11   let ciclo = timestamp mod 216 in  
12   match ciclo with  
13   | _ when ciclo < 90 -> "en-rojo"  
14   | _ when ciclo < 210 -> "en-verde"  
15   | _ -> "en-amarillo"  
16  
17 (*  
18 =====  
19 FUNCIÓN: auditoria  
20 NATURALEZA: Impura (produce efectos secundarios: imprime en terminal)  
21 ESTRATEGIA: Selección por condición (compara colores mediante if)  
22 IMPACTO: No destructiva (no modifica estructuras en memoria)  
23 =====  
24 *)  
25  
26 let auditoria timestamp =
```

Output

```
Tiempo 90: cambio de en-rojo a en-verde  
Tiempo 210: cambio de en-verde a en-amarillo  
val timer : int -> string = <fun>  
val auditoria : int -> unit = <fun>
```

Sin haber escrito ninguna anotación de tipos, el compilador dedujo solo que:

- Timer recibe un int (el timestamp) y devuelve un string (el color)
- Auditoria recibe un int y devuelve unit (no devuelve valor, solo imprime)

Lo dedujo analizando cómo se usan los valores dentro de las funciones: como `timestamp mod 216` usa mod, infiere que es entero. Como devuelve "en-rojo" entre comillas, infiere que es string.

Pregunta 2

OCaml es un lenguaje orientado a expresiones, lo que significa que las estructuras de control producen valores. En nuestro programa utilizamos la construcción `match ... with` para determinar el estado del semáforo según el instante de tiempo recibido:

```
match ciclo with  
| _ when ciclo < 90 -> "en-rojo"  
| _ when ciclo < 210 -> "en-verde"  
| _ -> "en-amarillo"
```

Esta expresión devuelve directamente una cadena de caracteres correspondiente al estado calculado.

Este comportamiento es similar al de Lisp, donde construcciones como `if` o `cond` también son expresiones que devuelven valores. De hecho, el fragmento anterior es conceptualmente equivalente a la siguiente expresión en Lisp:

```
(cond
  (< ciclo 90) "en-rojo")
(< ciclo 210) "en-verde")
(t "en-amarillo"))
```

La principal diferencia observada es sintáctica. Mientras que Lisp suele utilizar expresiones `cond` para seleccionar entre alternativas, OCaml utiliza `match ... with`, una construcción diseñada para el análisis de casos y patrones. En este ejercicio, `match ... with` permitió representar claramente los distintos estados del semáforo dentro de una única expresión que devuelve el resultado correspondiente.

Justificación de la librería seleccionada: local-time

De las dos opciones propuestas para la Fase 2, se optó por integrar `local-time` en lugar de `cl-json`, por las siguientes razones:

1. Conexión directa con un requerimiento ya existente y con su propósito declarado.

El Requerimiento 3 (Sistema de Auditoría) tiene como objetivo explícito permitir la "reconstrucción histórica de estados del semáforo" para análisis forense. Un timestamp en formato Unix (por ejemplo, 1718380800) cumple la función pero no es legible para una persona que tenga que auditar el sistema; una fecha como 2026-05-16 14:30:00 sí lo es. Integrar `local-time`, entonces, no es solo "cumplir con la Fase 2", sino que mejora directamente la utilidad real de una función que el propio enunciado ya había definido con un propósito claro. Con `cl-json`, en cambio, la mejora es sobre un aspecto secundario (de dónde provienen tres constantes numéricas), que no cambia la utilidad final del sistema para el usuario o el auditor.

2. Menor impacto sobre las funciones ya implementadas en la Fase 1.

Integrar `local-time` implicó modificar únicamente auditoria (y luego informe), agregando una transformación sobre el timestamp antes de formatear el mensaje de salida. El resto de las funciones del sistema (`timer`, `transicion`, `duracion-ciclo`, `ciclos-por-tiempo`, `distribucion-hora`, etc.) quedaron exactamente igual.

Integrar `cl-json`, en cambio, hubiese requerido redefinir de base prácticamente toda la estructura del programa. Cargar un archivo `.json` con los valores de rojo, verde y amarillo implica obtener esos tres valores en algún punto del programa y luego hacerlos llegar a todas las funciones que hoy dependen de esas duraciones: `timer` y `timer2` (que actualmente calculan el color a partir de constantes fijas dentro del `cond`), `duracion-ciclo`, `ciclos-por-tiempo` (que tiene hardcodeado el 216), `distribucion-hora` y

distribucion-hora2, entre otras. Esto significa que funciones que hoy reciben un solo argumento (como timer, que solo recibe el timestamp) pasarían a necesitar también rojo/verde/amarillo como parámetros adicionales, o bien depender de un valor leído desde el archivo de configuración que tendría que propagarse a través de toda la cadena de llamadas (auditoria llama a timer, informe llama a timer, etc.). En la práctica, hubiese significado rediseñar las firmas de la mayoría de las funciones del Requerimiento 1 al 6, en lugar de agregar una funcionalidad nueva sobre lo ya construido.

3. Consistencia con el criterio de "evitar hardcodeo donde la consigna lo permite".

Como se explicó en los fundamentos del diseño, el hardcodeo de valores (como el 216 en ciclos-por-tiempo) se mantuvo únicamente en los casos donde la consigna de la Fase 1 fijaba explícitamente la firma de la función y no dejaba margen para parametrizar. Las duraciones de rojo/verde/amarillo, en cambio, ya estaban resueltas como parámetros en las funciones que más lo necesitaban (duracion-ciclo, distribucion-hora, distribucion-hora2), que reciben esos valores como argumentos en lugar de tenerlos fijos en el cuerpo. Por lo tanto, el problema que cl-json viene a resolver, tiempos de temporización fijos en el código, ya estaba parcialmente mitigado por decisiones tomadas en la Fase 1, mientras que adoptarlo igualmente hubiese implicado el rediseño descrito en el punto anterior a cambio de una mejora parcial.

4. Curva de aprendizaje acorde al objetivo de "autonomía con mínima asistencia docente".

local-time ofrece una API acotada y bien documentada para el caso de uso puntual que se necesitaba (unix-to-timestamp + format-timestring), lo que permitió resolver la integración con Quicklisp (incluyendo los problemas de compatibilidad con CLISP y el conflicto de nombres con SBCL, documentados en la bitácora) sin que la complejidad de la librería en sí misma se sumara a la complejidad ya existente del entorno.

BITÁCORA DE DEPURACIÓN

1: Interpretación incorrecta de las transiciones válidas del semáforo

Problema encontrado

Al implementar la función **transición**, la consigna indicaba que debía devolverse **'accion-por-defecto'** cuando la transición no fuera válida. Sin embargo, la especificación no definía explícitamente qué transiciones debían considerarse válidas o inválidas.

Inicialmente se asumió que cualquier cambio entre los colores rojo, amarillo y verde era válido, por lo que no existía un criterio claro para determinar cuándo retornar 'accion-por-defecto'. Esto dejó la función sin lógica condicional real: el cuerpo quedó incompleto, con comentarios en lugar de código funcional, ya que no era posible avanzar sin conocer las reglas de validez.

Causa

La ambigüedad provenía de una interpretación incompleta de los requisitos funcionales. La especificación mencionaba transiciones inválidas, pero no describía la secuencia permitida de estados. En términos técnicos, sin una estructura condicional (cond por ejemplo) que validara el par (color-actual, cambiar-a), la función era incapaz de discriminar entre una transición válida e inválida, haciendo imposible implementar el retorno correcto.

Proceso de depuración

Se consultó al profesor mediante el aula virtual para aclarar el comportamiento esperado. Paralelamente, en el grupo de comunicación entre equipos surgió un debate adicional: se planteó la posibilidad de que las entradas de la función no se limitaran a rojo, amarillo y verde, sino que pudieran recibirse otros valores de color no contemplados. Bajo esa hipótesis, una transición inválida podría referirse simplemente a un color de entrada desconocido, independientemente de la secuencia. Sin embargo, esta interpretación no pudo confirmarse sin consultar al docente.

La respuesta fue:

"Al ser un semáforo vial, hay solo una secuencia cíclica válida."

A partir de esta aclaración, y asumiendo en rigor que los únicos colores posibles como entrada son rojo, amarillo y verde, se descartó la hipótesis de entradas arbitrarias y se establecieron las siguientes como las únicas transiciones permitidas:

rojo -> verde

verde -> amarillo

amarillo -> rojo

Cualquier otra combinación debía considerarse inválida y disparar el retorno de 'accion-por-defecto'.

Solución

Se redefinió la función para verificar explícitamente la secuencia cíclica del semáforo y devolver 'accion-por-defecto' cuando la transición no respetara el ciclo establecido.

Evidencia

Captura de la consigna original.

Requerimiento 1: Estados de Transición

Implemente la función `transicion` que modele el cambio de estados del semáforo:

Especificación:

- **Entrada:** `color-actual` (símbolo: en-rojo, en-amarillo, en-verde) y `cambiar-a` (símbolo del color destino: rojo, amarillo, verde)
- **Salida:** devuelve una lista con el estado y una acción a realizar, esta última como literal `"cambiar-a-<color>"`.
- **Comportamiento:** Por defecto, retorna color actual y 'accion-por-defecto' si la transición no es válida

Ejemplo esperado:

`(transicion 'en-rojo 'verde) → ('en-rojo "cambiar-a-verde")`

Captura de la consulta enviada al profesor y la respuesta.

29 de mayo

17:20

Buenas tardes, profesor. Tengo una consulta respecto al Requerimiento 1 del core en Lisp ("Estados de Transición").

En la especificación se indica que, por defecto, la función debe retornar el color actual y 'accion-por-defecto' si la transición no es válida. Sin embargo, no encontré en la consigna un apartado donde se especifique qué transiciones se consideran válidas o inválidas entre colores.

Por ejemplo, no me queda claro si cualquier cambio entre colores ('rojo', 'amarillo', 'verde') debería considerarse válido, o si debe respetarse una secuencia específica de transición del semáforo.

¿Podría aclararme a qué se refiere exactamente con "transición no válida"?

Desde ya, muchas gracias.

 **Matías Mascazzini** 19:12 ○

Al ser un semaforo vial, hay solo una secuencia ciclica valida

Con respecto al tiempo, ustedes decidan desde que fecha empieza a funcionar

Captura del código con lógica incompleta y comentarios previo a la aclaración

```
(defun transicion (color-actual cambiar-a)
  (list color-actual ... )
  ;definir que es una transicion invalida y una valida para resolver.
  ;Devuelve una lista con color actual y un string
```

2: Ambigüedad en el origen temporal del ciclo del semáforo

Descripción del problema

Durante la implementación de la función encargada de determinar el color del semáforo a partir de un tiempo Unix (epoch), surgió una duda respecto al instante inicial del ciclo.

La consigna especificaba las duraciones de cada estado (rojo = 90 s, amarillo = 6 s, verde = 120 s), pero no indicaba desde qué momento debía comenzar a contarse el primer ciclo.

Inicialmente la implementación fue dejada incompleta hasta confirmar la interpretación correcta del requerimiento.

Causa

La especificación indicaba que la entrada era un tiempo Unix, pero no establecía un instante de referencia para el comienzo del ciclo del semáforo. Sin este dato, el mismo timestamp podía asociarse a colores distintos según la fecha elegida como inicio.

Proceso de depuración

Se consultó al profesor para determinar si debía asumirse que el semáforo comenzaba en el instante 0 del tiempo Unix y si el ciclo debía repetirse indefinidamente.

La respuesta fue:

"Con respecto al tiempo, ustedes decidan desde qué fecha empieza a funcionar. Sí, se asume un ciclo continuo."

Solución

Se decidió utilizar el instante 0 como referencia inicial por ser el origen natural del tiempo Unix y porque permite que el algoritmo funcione correctamente para cualquier entero recibido como parámetro.

Además, se mantuvo la secuencia de estados definida previamente para el semáforo:

rojo -> verde
verde -> amarillo
amarillo -> rojo

y no el orden en que los colores aparecían mencionados dentro de la consigna.

De esta forma, el color correspondiente a cualquier timestamp se obtiene calculando el resto de la división entre el tiempo recibido y la duración total del ciclo, evaluando luego en qué tramo del ciclo se encuentra dicho instante.

Captura de la consulta enviada al profesor y la respuesta

17:37

Buenas tardes, profesor.

Quería realizar también otra consulta respecto al Requerimiento 2 (Timer).

Entiendo que la función debe recibir un tiempo Unix/epoch y determinar qué color del semáforo corresponde en ese instante según las duraciones indicadas (rojo 90 s, amarillo 6 s y verde 120 s). Mi duda es sobre el punto de referencia temporal del ciclo.

¿Debe asumirse que el semáforo comienza en el instante '0' (epoch 0) en rojo y que, a partir de allí, continúa cíclicamente siguiendo la secuencia indicada (rojo, amarillo, verde... nuevamente rojo)? Es decir, asumir un ciclo continuo desde ese punto de inicio para determinar el color correspondiente a cualquier timestamp.

Desde ya, muchas gracias!

19:12

 **Matías Mascazzini**

Al ser un semaforo vial, hay solo una secuencia ciclica valida

Con respecto al tiempo, ustedes decidan desde que fecha empieza a funcionar

19:13

 **Matías Mascazzini**

si, se asume un ciclo continuo

3. Diseño de la interfaz de la función auditoria

Descripción del problema Al implementar auditoria surgió la duda de qué argumentos debía recibir para poder registrar el cambio de estado correctamente.

Causa La primera idea fue pasarle dos timestamps, time-anterior y time-actual, y que la función consultara timer con cada uno para comparar los colores. Funcionaba, pero se sentía redundante.

Proceso de depuración Pensándolo un poco más, el semáforo cambia en instantes determinados que ya conocemos gracias a timer. Si evaluamos (timer t) y (timer (- t 1)) y los colores difieren, ese es exactamente el momento del cambio. Como timer es pura y el ciclo es predecible, el color anterior siempre se puede obtener restando 1 al timestamp actual, sin necesidad de que el caller lo calcule por su cuenta.

Solución Se simplificó la interfaz para que auditoria reciba solo el timestamp actual. El color anterior se deriva internamente con (- timestamp 1), y la función solo imprime cuando detecta un cambio real. Menos argumentos, misma información

Captura del Código Anterior y del Código Actual

```
;; =====
;; FUNCIÓN: auditoria
;; NATURALEZA: Impura (produce efectos secundarios: imprime en terminal)
;; ESTRATEGIA: Selección por casos (compara colores mediante cond)
;; IMPACTO: No destructiva (No modifica estructuras en memoria)
;; =====

(defun auditoria (time-anterior time-actual)
  (let ((color-anterior (timer time-anterior))
        (color-actual (timer time-actual)))
    (cond
      ((not (equal color-anterior color-actual))
       (format t "Tiempo ~A: la luz ha cambiado de ~A a ~A~%"
               time-actual
               color-anterior
               color-actual))))
  )
```

```
(defun auditoria (epoch)
  (let ((color-anterior (timer (- epoch 1)))
        (color-actual (timer epoch)))
    (cond
      ((not (equal color-anterior color-actual))
       (format t "Tiempo ~A: la luz ha cambiado de ~A a ~A~%"
               epoch
               color-anterior
               color-actual))))
  )
```

4. Incompatibilidad de CLISP con Quicklisp

Descripción del problema

Para la integración de la librería local-time mediante Quicklisp, se intentó instalar y ejecutar Quicklisp desde el entorno utilizado en la cátedra, CLISP 2.49, sin éxito.

Causa Quicklisp no es compatible con CLISP. Al intentar cargar el instalador desde PowerShell con distintos comandos, el proceso fallaba con el error `ERROR_DIRECTORY: The directory name is invalid`, sin llegar a completar la instalación.

Proceso de depuración Se realizaron varias pruebas desde PowerShell intentando distintas formas de invocar CLISP con el archivo `quicklisp.lisp`, descargado desde la página oficial. Ninguna de las variantes probadas logró superar el error de compatibilidad.

Solución

Se descargó e instaló SBCL (Steel Bank Common Lisp), implementación de Common Lisp compatible con Quicklisp. Para la integración con el entorno de desarrollo, se configuró Visual Studio Code con la extensión Alive, que provee un REPL conectado directamente

a SBCL. Desde ese entorno se pudo instalar Quicklisp correctamente, cargar la librería local-time y verificar su funcionamiento con éxito.

Captura de Pantalla de la instalación de QUICKLISP e Instalación de LocalTime

```
Windows PowerShell
Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\WINDOWS\system32> sbcl --load "C:\Users\Yle\Downloads\quicklisp.lisp"
This is SBCL 2.6.5, an implementation of ANSI Common Lisp.
More information about SBCL is available at <http://www.sbcl.org/>.

SBCL is free software, provided as is, with absolutely no warranty.
It is mostly in the public domain; some portions are provided under
BSD-style licenses. See the CREDITS and COPYING files in the
distribution for more information.

==== quicklisp quickstart 2015-01-28 loaded ====

To continue with installation, evaluate: (quicklisp-quickstart:install)

For installation options, evaluate: (quicklisp-quickstart:help)

* (quicklisp-quickstart:install)
; Fetching #<URL "http://beta.quicklisp.org/client/quicklisp.sexp">
; 0.82KB
=====
839 bytes in 0.00 seconds (962.79KB/sec)
; Fetching #<URL "http://beta.quicklisp.org/client/2021-02-13/quicklisp.tar">
; 260.00KB
=====
266,240 bytes in 0.14 seconds (1898.99KB/sec)
; Fetching #<URL "http://beta.quicklisp.org/client/2021-02-11/setup.lisp">
; 4.94KB
=====
5,057 bytes in 0.00 seconds (5709.22KB/sec)
; Fetching #<URL "http://beta.quicklisp.org/asdf/3.2.1/asdf.lisp">
; 628.18KB
=====
643,253 bytes in 0.26 seconds (2379.09KB/sec)
; Fetching #<URL "http://beta.quicklisp.org/dist/quicklisp.txt">
; 0.40KB
=====
408 bytes in 0.00 seconds (328.74KB/sec)
Installing dist "quicklisp" version "2026-01-01".
; Fetching #<URL "http://beta.quicklisp.org/dist/quicklisp/2026-01-01/releases.txt">
; 564.04KB
=====
577,575 bytes in 0.22 seconds (2601.71KB/sec)
; Fetching #<URL "http://beta.quicklisp.org/dist/quicklisp/2026-01-01/systems.txt">
; 450.96KB
=====
469,975 bytes in 0.23 seconds (2033.04KB/sec)

==== quicklisp installed ====

To load a system, use: (ql:quickload "system-name")

To find systems, use: (ql:system-apropos "term")

To load Quicklisp every time you start Lisp, use: (ql:add-to-init-file)

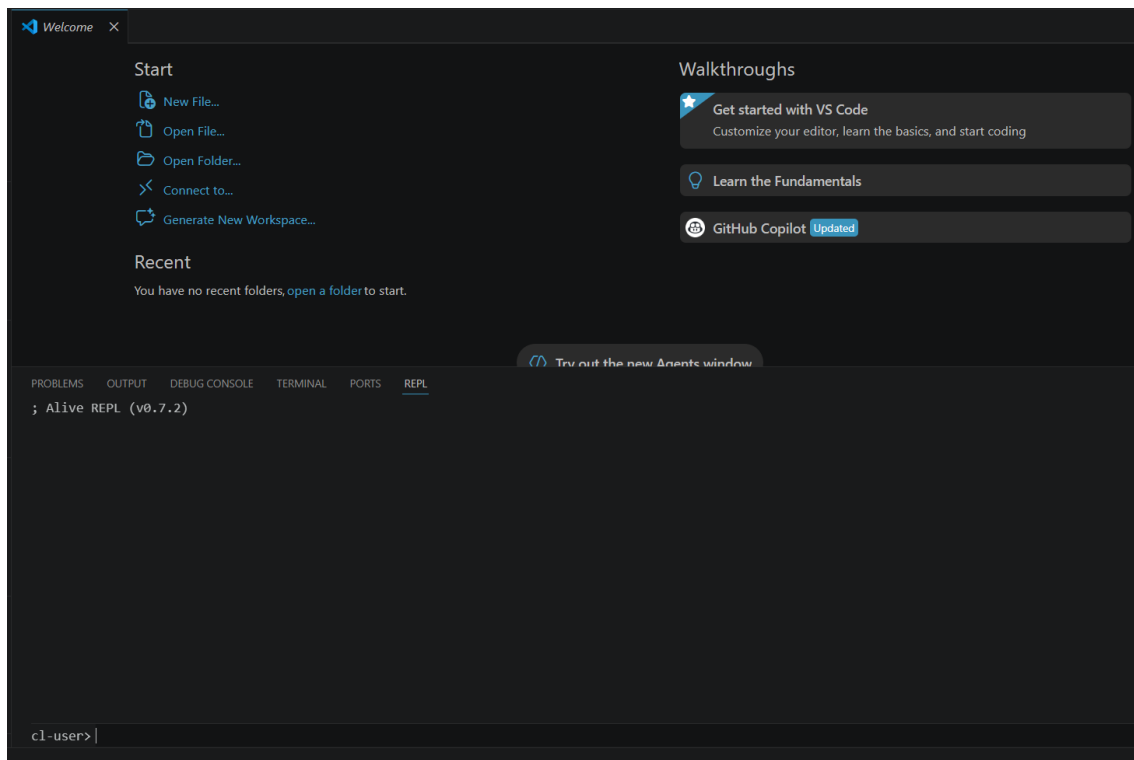
For more information, see http://www.quicklisp.org/beta/

NIL
* (ql:add-to-init-file)
I will append the following lines to #P"C:/Users/Yle/.sbclrc":

;;; The following lines added by ql:add-to-init-file:

* (ql:quickload "local-time")
To load "local-time":
  Load 2 ASDF systems:
    asdf uiop
  Install 1 Quicklisp release:
    local-time
; Fetching #<URL "http://beta.quicklisp.org/archive/local-time/2026-01-01/local-time-20260101-git.tgz">
; 550.30KB
=====
563,503 bytes in 0.25 seconds (2158.71KB/sec)
; Loading "local-time"
[package local-time].....
("local-time")
* |
```

ALIVE REPL en Visual Estudio Code



5. Conflicto de nombre con timer en SBCL

Descripción del problema

Al trasladar el código al entorno SBCL para la integración con Quicklisp, la definición de la función timer generaba el siguiente error:

"Lock on package SB-EXT violated when proclaiming TIMER as a function while in package COMMON-LISP-USER."

Causa

El nombre timer está asociado a un símbolo del paquete SB-EXT de SBCL, utilizado por su sistema de temporizadores. Debido al mecanismo de *package locks*, SBCL impide redefinir accidentalmente símbolos pertenecientes a paquetes del sistema. En CLISP 2.49 este conflicto no se presenta, por lo que el código funcionaba correctamente en el entorno utilizado durante el desarrollo inicial.

Proceso de depuración

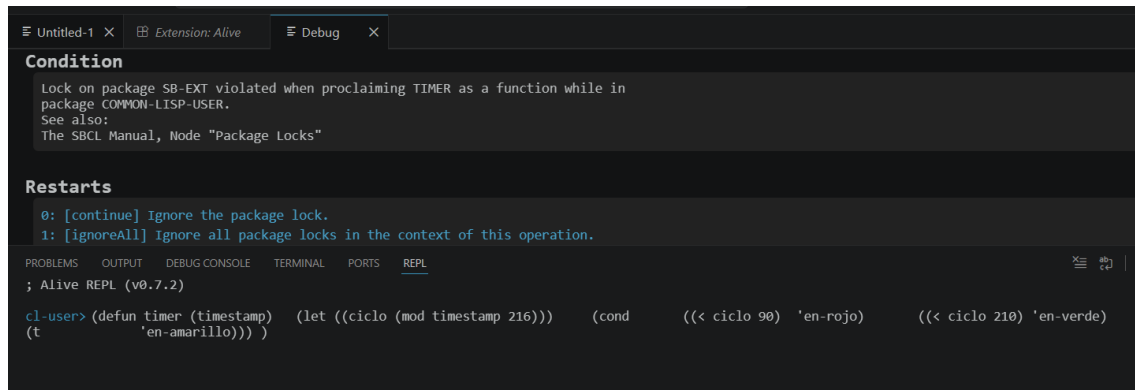
Se verificó que el error estaba relacionado con un conflicto de nombres y no con la lógica de la función. Posteriormente se consultó la documentación oficial de SBCL sobre el mecanismo de *package locks*, confirmando que el símbolo timer pertenecía a un paquete protegido del sistema.

Solución

Se agregó la instrucción: (**shadow 'timer**)

antes de la definición de la función. Esto crea un símbolo local timer en el paquete actual, ocultando el símbolo homónimo de SB-EXT. De esta manera se resolvió el conflicto sin necesidad de renombrar la función ni modificar el resto del código.

Captura del ERROR al no crear el símbolo local



6. Errores de sintaxis en la traducción de Common Lisp a OCaml

Descripción del problema Durante el proceso de traducción de las funciones de Common Lisp a OCaml, se cometieron varios errores de sintaxis por desconocimiento del lenguaje.

Causa Los errores surgieron de intentar mantener la misma estructura y nombres de variables que en la versión Lisp original, sin considerar las diferencias sintácticas de OCaml. En primer lugar, se utilizó != para expresar desigualdad, cuando en OCaml el operador correcto es <>. En segundo lugar, se emplearon guiones (-) en los nombres de variables como color-anterior y color-actual, siendo que OCaml interpreta el guion como el operador de resta y exige el uso de guion bajo (_) en su lugar. Por último, se escribió printf.printf en minúscula, cuando la forma correcta es Printf.printf, ya que en OCaml los módulos se escriben con la primera letra en mayúscula.

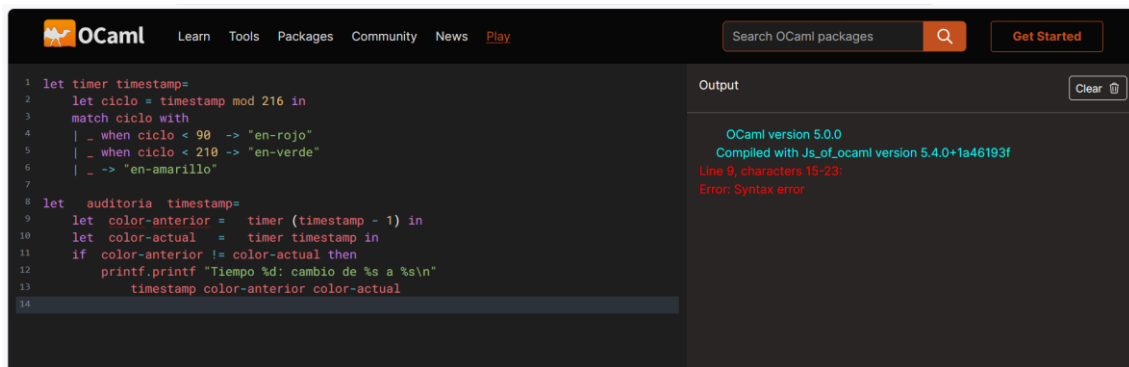
Proceso de depuración Se identificaron los errores a partir de los mensajes del compilador, que señalaba la línea y el tipo de error. Esto permitió corregir cada problema de forma puntual sin modificar la lógica del código.

Solución Se reemplazó != por <>, los guiones en nombres de variables por guion bajo (_), y se corrigió la capitalización de Printf.printf.

Primera versión del código con errores de sintaxis

```
let auditoria timestamp=  
  let color-anterior = timer (timestamp - 1) in  
  let color-actual = timer timestamp in  
  if color-anterior != color-actual then  
    printf.printf "Tiempo %d: cambio de %s a %s\n"  
      timestamp color-anterior color-actual
```

Detección de errores por parte del compilador



The screenshot shows the OCaml online compiler interface. The code editor on the left contains the following code:

```
1 let timer timestamp=  
2   let ciclo = timestamp mod 216 in  
3   match ciclo with  
4   | _ when ciclo < 90 -> "en-rojo"  
5   | _ when ciclo < 210 -> "en-verde"  
6   | _ -> "en-amarillo"  
7  
8 let auditoria timestamp=  
9   let color-anterior = timer (timestamp - 1) in  
10  let color-actual = timer timestamp in  
11  if color-anterior != color-actual then  
12    printf.printf "Tiempo %d: cambio de %s a %s\n"  
13      timestamp color-anterior color-actual  
14
```

The output panel on the right shows the following message:

```
OCaml version 5.0.0  
Compiled with Js_of_ocaml version 5.4.0+1a46193f  
Line 9, characters 15-23:  
Error: Syntax error
```

7. Cálculo incorrecto de la distribución horaria en distribucion-hora

Descripción del problema

Al implementar distribucion-hora, la función calculaba los porcentajes de tiempo en rojo, verde y amarillo durante una hora, pero presentaba dos problemas: por un lado los resultados no se correspondían con la realidad del semáforo, y por otro la función no era reutilizable para distintas reglas de negocio.

Causa

La primera versión definía rojo, verde y amarillo como valores fijos dentro de un `let*` (90, 120 y 6 segundos respectivamente), en lugar de recibirlos como parámetros. Esto hacía que la función solo sirviera para las reglas de negocio actuales, cuando la consigna pide que funcione "dadas ciertas reglas de negocio" o "según las reglas actuales", es decir, que los tiempos de cada fase sean variables de entrada. Además, calculaba el porcentaje de cada fase directamente sobre la duración de un solo ciclo ((/ rojo total) * 100, etc.), asumiendo que esa proporción se mantenía igual a lo largo de toda la hora.

Proceso de depuración

Primero se identificó que al fijar los valores adentro de la función, cualquier cambio en las reglas de negocio (por ejemplo, otros tiempos de rojo/verde/amarillo) obligaría a reescribir la función en lugar de simplemente pasarle otros argumentos. Por eso se decidió convertir rojo, verde y amarillo en parámetros de la función.

Resuelto eso, se pensó en el segundo problema: una hora (3600 segundos) no es necesariamente un múltiplo exacto de la duración del ciclo. Con rojo=90, verde=120, amarillo=6 (valores de ejemplo según las reglas actuales), el ciclo dura 216 segundos, y en una hora entran 16 ciclos completos (3456 segundos), quedando 144 segundos sobrantes que no completan un ciclo entero. Esos segundos sobrantes también forman parte de la hora y deben repartirse según el orden real del ciclo (primero rojo, luego verde, luego amarillo), no de forma proporcional.

Para obtener la cantidad de ciclos completos en la hora se consideró reutilizar ciclos-por-tiempo, pero esa función tiene hardcoded el valor 216 (la duración del ciclo según las reglas actuales), por lo que su resultado dejaría de ser correcto si se la llamara con otras reglas de negocio. Como `distribucion-hora` ya recibe rojo, verde y amarillo como parámetros, se optó por calcular la duración del ciclo en base a esos parámetros (reutilizando `duracion-ciclo`) y obtener la cantidad de ciclos completos dividiendo 3600 por esa duración.

Solución

Se reformuló `distribucion-hora` para que reciba rojo, verde y amarillo como parámetros (volviéndola válida para cualquier conjunto de reglas de negocio, no solo las actuales). La duración del ciclo se obtiene reutilizando `duracion-ciclo`, y a partir de ella se calcula directamente la cantidad de ciclos completos en una hora ((`truncate (/ 3600 duracion)`)). Con eso se calcula el tiempo acumulado de cada fase en esos ciclos completos, se obtiene el tiempo sobrante (3600 menos lo ya cubierto), y ese sobrante se reparte secuencialmente entre rojo, verde y amarillo según el orden del ciclo. Recién con esos totales reales se calculan los porcentajes sobre 3600 segundos, en lugar de sobre la duración de un solo ciclo.

Captura del Código Anterior y del Código Actual

```
;; =====  
;; FUNCIÓN: distribucion-hora  
;; NATURALEZA: Impura (produce efectos secundarios: imprime en terminal)  
;; ESTRATEGIA: Calculo aritmetico, seguido de impresión formateada.  
;; IMPACTO: No destructiva (no modifica estructuras en memoria)  
;; =====  
  
(defun distribucion-hora ()  
  (let* ((rojo 90)  
         (verde 120)  
         (amarillo 6)  
         (total (+ rojo verde amarillo)))  
    (format t "=== Distribucion Temporal (1 hora) ===~%")  
    (format t "Rojo: ~,2F%~%" (* (/ (float rojo) total) 100))  
    (format t "Verde: ~,2F%~%" (* (/ (float verde) total) 100))  
    (format t "Amarillo: ~,2F%~%" (* (/ (float amarillo) total) 100))  
  )  
)
```

```
;; =====
;; FUNCIÓN: distribucion-hora
;; NATURALEZA: Impura (produce efectos secundarios: imprime en terminal)
;; ESTRATEGIA: Calculos aritmeticos, seguido de impresión formateada.
;; IMPACTO: No destructiva (no modifica estructuras en memoria)
;; =====
(defun distribucion-hora (rojo verde amarillo)
  (let* ((duracion (duracion-ciclo rojo verde amarillo))
        (ciclos (truncate (/ 3600 duracion)))
        (cubierto (* ciclos duracion))
        (resto (- 3600 cubierto))
        (rojo-extra (min resto rojo))
        (resto-verde (- resto rojo-extra))
        (verde-extra (min resto-verde verde))
        (resto-amarillo (- resto-verde verde-extra))
        (amarillo-extra (min resto-amarillo amarillo))
        (rojo-total (+ (* ciclos rojo) rojo-extra))
        (verde-total (+ (* ciclos verde) verde-extra))
        (amarillo-total (+ (* ciclos amarillo) amarillo-extra)))
    (format t "=== Distribucion Temporal (1 hora) ===~%")
    (format t "Rojo: ~,2F%~%" (* (/ (float rojo-total) 3600) 100))
    (format t "Verde: ~,2F%~%" (* (/ (float verde-total) 3600) 100))
    (format t "Amarillo: ~,2F%~%" (* (/ (float amarillo-total) 3600) 100))
  )
)

;; Calcula el % de tiempo en rojo, verde y amarillo durante 1 hora,
;; repartiendo secuencialmente el tiempo sobrante de los ciclos.
```

8. Refactorización de distribución-hora

Descripción del problema

La implementación de `distribucion-hora` (y su versión `distribucion-hora2` para la segunda iteración, con 6 fases en vez de 3) repetía manualmente, por cada fase, el mismo patrón: calcular `*-extra` con `min`, restar ese extra del resto para obtener el siguiente resto, y luego sumar `(* ciclos duracion-fase) + extra` para obtener el total. Con 3 fases (rojo, verde, amarillo) esto ya generaba 9 variables intermedias (`rojo-extra`, `resto-verde`, `verde-extra`, `resto-amarillo`, `amarillo-extra`, `rojo-total`, `verde-total`, `amarillo-total`, más `duracion/ciclos/resto`). Al pasar a `distribucion-hora2`, donde `timer2` agrega los 3 estados intermitentes (6 fases en total: rojo, rojo-intermitente, verde, verde-intermitente, amarillo, amarillo-intermitente), el mismo patrón hubiese duplicado la cantidad de variables a casi 20, haciendo la función excesivamente larga y repetitiva.

Causa

El reparto secuencial del tiempo sobrante entre las fases era, en esencia, la misma operación repetida N veces (una por fase), pero estaba escrita de forma completamente manual y desenrollada, sin abstraer el patrón común. Esto hacía que la

función no escalara: agregar o quitar fases (como pasó al pasar de 3 a 6 fases en la segunda iteración) implicaba reescribir gran parte del cuerpo de la función.

Proceso de depuración

Se identificó que el patrón extra = (min resto duracion-fase) seguido de resto = resto - extra, aplicado secuencialmente sobre una lista ordenada de duraciones, es exactamente una recursión: se procesa la primera duración de la lista, se calcula su extra, y se llama recursivamente con el resto actualizado y el resto de la lista. Esto permitió extraer esa lógica a una función auxiliar pura, repartir-resto, que recibe el resto inicial y la lista de duraciones (en el orden real del ciclo) y devuelve la lista de extras correspondientes, sin importar cuántas fases tenga la lista.

Una vez separados los extras, faltaba combinar cada extra con su duracion correspondiente para obtener el total de cada fase (($\times$ ciclos duracion) + extra), y luego combinar cada total con su nombre para imprimirlo. Ambas operaciones son "aplicar la misma función a cada par de elementos de dos listas en paralelo", que es exactamente lo que hace mapcar con múltiples listas.

Solución

Se extrajo el reparto secuencial a una función auxiliar repartir-resto, pura y recursiva, que generaliza el encadenado de min/resta para una lista de cualquier longitud. distribucion-hora quedó reducida a: armar la lista de duraciones de las fases en orden, llamar a repartir-resto para obtener los extras, y usar mapcar dos veces (una para calcular los totales combinando duraciones y extras, y otra para imprimir combinando nombres y totales). Esto eliminó las variables intermedias nombradas a mano, y permite que distribucion-hora2 (6 fases) reutilice exactamente el mismo repartir-resto, sin duplicar lógica: solo cambia la lista de duraciones y la lista de nombres que se le pasan.

Código anterior y Código nuevo

```
;; =====
;; FUNCIÓN: distribucion-hora
;; NATURALEZA: Impura (produce efectos secundarios: imprime en terminal)
;; ESTRATEGIA: Calculos aritmeticos, seguido de impresión formateada.
;; IMPACTO: No destructiva (no modifica estructuras en memoria)
;; =====
(defun distribucion-hora (rojo verde amarillo)
  (let* ((duracion (duracion-ciclo rojo verde amarillo))
        (ciclos (truncate (/ 3600 duracion)))
        (cubierto (* ciclos duracion))
        (resto (- 3600 cubierto))
        (rojo-extra (min resto rojo))
        (resto-verde (- resto rojo-extra))
        (verde-extra (min resto-verde verde))
        (resto-amarillo (- resto-verde verde-extra))
        (amarillo-extra (min resto-amarillo amarillo))
        (rojo-total (+ (* ciclos rojo) rojo-extra))
        (verde-total (+ (* ciclos verde) verde-extra))
        (amarillo-total (+ (* ciclos amarillo) amarillo-extra)))
    (format t "=== Distribucion Temporal (1 hora) ===~%")
    (format t "Rojo: ~,2F%-~%" (* (/ (float rojo-total) 3600) 100))
    (format t "Verde: ~,2F%-~%" (* (/ (float verde-total) 3600) 100))
    (format t "Amarillo: ~,2F%-~%" (* (/ (float amarillo-total) 3600) 100))
  )
)

;; Calcula el % de tiempo en rojo, verde y amarillo durante 1 hora,
;; repartiendo secuencialmente el tiempo sobrante de los ciclos.
```

```
;; =====
;; FUNCIÓN: repartir-resto
;; NATURALEZA: Pura (a misma entrada, misma salida)
;; ESTRATEGIA: Recursion de Cola
;; IMPACTO: No destructiva
;; =====

(defun repartir-resto (resto duraciones)
  (cond
    ((null duraciones) nil)
    (t (let ((extra (min resto (car duraciones))))
         (cons extra
               (repartir-resto (- resto extra) (cdr duraciones)))
         )
    )
  )

;Funcion Auxiliar de distribucion-hora

;; =====
;; FUNCIÓN: distribucion-hora
;; NATURALEZA: Impura (produce efectos secundarios: imprime en terminal)
;; ESTRATEGIA: Funcion de orden superior (Mapcar) y Calculos aritmeticos
;; IMPACTO: No destructiva (no modifica estructuras en memoria)
;; =====

(defun distribucion-hora (rojo verde amarillo)
  (let* ((duracion (duracion-ciclo rojo verde amarillo))
        (ciclos (truncate (/ 3600 duracion)))
        (resto (- 3600 (* ciclos duracion)))
        (duraciones (list rojo verde amarillo))
        (extras (repartir-resto resto duraciones))

        (nombres '("Rojo" "Verde" "Amarillo"))

        (totales (mapcar (lambda (d e) (+ (* ciclos d) e)) duraciones extras)))
    (format t "=== Distribucion Temporal (1 hora) ===~%"
      (mapcar (lambda (nombre total)
                (format t "~A: ~,2F%%~%" nombre (* (/ (float total) 3600) 100)))
              nombres totales)
    )
  )
)
```

CONCLUSION

Sobre OCaml

La experiencia de estudio y utilización de OCaml resultó interesante y, en una primera aproximación, algo confusa. Una de las dificultades iniciales fue acostumbrarse a una sintaxis que no utiliza delimitadores explícitos para marcar el final de una función o de un bloque, como ocurre en otros lenguajes. En particular, viniendo de Lisp, donde la estructura del programa queda claramente delimitada mediante paréntesis (), al principio resultó menos evidente identificar dónde terminaba cada expresión o definición. También generó cierta confusión el uso de elementos de la biblioteca estándar cuyos nombres distinguen entre mayúsculas y minúsculas, como el caso de `Printf.printf`, donde el mismo nombre aparece tanto para el módulo como para la función con distinta capitalización, lo cual no resulta intuitivo para un usuario principiante.

Sin embargo, una vez comprendida la sintaxis, la implementación de las soluciones no presentó mayores dificultades. El hecho de que OCaml sea un lenguaje funcional en el que las expresiones producen valores recuerda mucho a Lisp, por lo que varios conceptos resultaron familiares. De hecho, la traducción de las soluciones desde Lisp hacia OCaml fue bastante directa, ya que ambos comparten muchas ideas

fundamentales relacionadas con la programación funcional y el uso de expresiones que retornan resultados.

Otro aspecto que inicialmente llamó la atención fue el uso de símbolos y patrones como `|` y `_` dentro de las expresiones `match`. Aunque al principio parecen extraños, terminan ofreciendo una representación muy clara de las distintas alternativas posibles. Las ramas de ejecución quedan delimitadas de forma visual y casi esquemática, facilitando la lectura y comprensión del código.

Otra diferencia práctica que encontramos fue la forma de probar los programas. Aunque OCaml dispone de entornos interactivos (REPL), el playground oficial de ocaml.org ejecutaba el archivo completo mediante la opción "Run", sin permitir la interacción continua que habíamos utilizado en Lisp. Por esta razón fue necesario incluir llamadas de ejemplo dentro del código para verificar y demostrar el funcionamiento de las funciones implementadas.

En conclusión, si bien OCaml posee algunas particularidades sintácticas que requieren un período de adaptación, encontramos que su enfoque funcional, la inferencia de tipos y la expresividad de construcciones como `match ... with` lo convierten en un lenguaje ordenado y agradable de utilizar. Además, la similitud conceptual con Lisp permitió que la transición entre ambos lenguajes fuera relativamente sencilla una vez comprendidas las diferencias sintácticas.

Sobre el proyecto en general

El desarrollo del Sistema de Semáforos Inteligentes resultó un desafío considerable, sobre todo en una primera instancia. La consigna, en un primer momento, generaba cierta incertidumbre porque no se parecía a los ejercicios trabajados durante la cursada: las restricciones de diseño obligaban a pensar la solución de una manera distinta a la habitual, y eso impuso una curva de entrada más empinada de lo esperado.

Sin embargo, a medida que avanzamos, el proyecto también dejó espacio para la creatividad del grupo, más allá de lo estrictamente pedido por la consigna. La incorporación de las funciones `universal-a-unix` y `ejecutar-sistema/ejecutar-sistema2`, no solicitadas explícitamente, nos permitió pensar el sistema no solo como una serie de funciones aisladas que resuelven requerimientos puntuales, sino como algo que podría aproximarse a un entorno real de ejecución continua, demostrando que el código desarrollado es funcional también en escenarios de uso prolongado y no solo ante llamadas individuales.

Por último, uno de los aprendizajes más valiosos del trabajo fue notar cuánto transfiere el conocimiento del paradigma funcional entre lenguajes. A pesar de no conocer OCaml previamente, haber comprendido en profundidad los conceptos de funciones

puras, inmutabilidad, recursión y expresiones que devuelven valores en Lisp permitió encarar la traducción a OCaml con una curva de aprendizaje muy rápida, enfocándonos casi exclusivamente en diferencias sintácticas y no en la lógica de la solución, que ya estaba resuelta conceptualmente. Esto reforzó la idea de que comprender un paradigma de programación es más valioso y duradero que dominar la sintaxis de un lenguaje específico.

BIBLIOGRAFÍA

CaesarCipher. (s.f.). *Unix timestamp / Epoch time explained*. CaesarCipher. <https://caesarcipher.org/learn/unix-timestamp-epoch-time-explained>

Common Lisp Libraries. (s.f.). *local-time*. Read the Docs. <https://common-lisp-libraries.readthedocs.io/local-time/>

Quicklisp Project. (s.f.). *Quicklisp beta*. Quicklisp. <https://www.quicklisp.org/beta/>

Steel Bank Common Lisp. (s.f.). *SBCL User Manual*. Secciones consultadas: *Package Locks* y *Timers*. SBCL. <https://www.sbcl.org/manual/index.html>

OCaml. (s.f.). *OCaml in Industry*. OCaml. <https://ocaml.org/industrial-users>

OCaml. (s.f.). *Why OCaml?* OCaml. <https://ocaml.org/about>